Diffusion Drafters for Speculative^2 Decoding: A Systems Study

Cashel Fitzgerald
Cornell University
cf566@cornell.edu

Tishya Khanna
Cornell University
tk732@cornell.edu

Abstract

Speculative decoding accelerates autoregressive language model inference by letting a cheap draft model propose multiple future tokens that a target model verifies in parallel. Speculative^2 Decoding (S^{2}D) goes further: while the target verifies one draft, a separate draft device precomputes continuations for likely verification outcomes. This naturally invites a diffusion drafter, since a block-diffusion model can in principle generate an entire draft block in parallel. We study that idea in two forms. First, we integrate conventional masked and block-diffusion language models into the S^{2}D draft interface. This negative result is the main lesson: practical block diffusion still requires many refinement forwards, often approaching the number of tokens in the block, so the drafter becomes the bottleneck. Our best traditional block-diffusion runs reached only about 20 tokens/s. Second, we prototype a DFlash-style hybrid that uses target activations from the verifier to produce a first draft block on S^{2}D cache misses, while the normal autoregressive S^{2}D drafter serves cache hits and fills the future speculation cache. This is architecturally more promising because DFlash drafts a block in a single conditioned pass, but our current small-model prototype remains limited by systems overhead: on Qwen3-8B with a Qwen3-0.6B S^{2}D drafter and `Qwen3-8B-DFlash-b16`, we measured 11.68 tokens/s, average accepted length 3.30, cache hit rate 0.60, DFlash seed latency 34.48 ms, and SSD tree-decode latency about 79 ms. We conclude that diffusion drafters are only useful for S^{2}D when draft latency is far below target verification latency, and that representative evaluation requires a larger verifier and less resource-constrained hardware.

1. Introduction

Autoregressive (AR) large language model inference is sequential: each generated token depends on the full prefix, and each step therefore requires another target-model forward pass. This is especially costly for verifier models in the tens of billions of parameters, where latency is dominated by memory movement and KV-cache traffic rather than by abundant GPU arithmetic. Speculative decoding (SD) reduces the number of serial target forwards by using a cheaper draft model to propose k future tokens and verifying those tokens in one target pass [2, 6]. The resulting accept/reject rule is lossless: the final sequence is distributed exactly as if it had been sampled from the target.

Speculative^2 Decoding (S^{2}D), implemented as Saguaro, attacks the next source of idle time [5]. In ordinary SD, the draft model waits while the target verifies, and the target then waits while the draft prepares the next block. S^{2}D places the draft model on separate hardware and asks it to predict likely verification outcomes while the target is busy. If the actual outcome is one of the precomputed branches, the next draft is ready immediately. The method is attractive because it can improve the throughput-latency frontier without changing the verifier or giving up exact sampling.

This report asks whether diffusion language models are a good replacement for the AR drafter inside S^{2}D. The motivation is straightforward. Discrete diffusion language models denoise masked tokens in parallel [1, 7], so a diffusion drafter appears to remove the inner AR bottleneck from the draft side. If one block forward could replace k serial AR draft steps, S^{2}D could use its separate draft GPU more effectively and cache likely future branches faster.

Our experiments show that this idea is more subtle. Conventional masked or block-diffusion drafters do not behave like a one-pass block oracle. To get usable draft quality, they require several denoising/refinement forwards and, in the quality regime, the number of refinement passes can approach the number of tokens generated in the block. In S^{2}D this cost is paid on the draft critical path and can dominate the very latency S^{2}D is meant to hide. In our implementation, the traditional block-diffusion backend peaked around 20 tokens/s and was not competitive with AR draft baselines.

DFlash changes the trade-off [3]. Rather than asking a standalone diffusion model to infer the next block from tokens alone, DFlash conditions a lightweight block-diffusion

drafter on hidden activations extracted from the target verifier. This target-conditioned design produces a whole speculative block in a single draft pass and has published speedups on Qwen3-8B and Qwen3-Coder-30B-A3B using SGLang [9]. We therefore extended our study beyond conventional block diffusion and implemented a DFlash-seeded S^{2}D hybrid.

Contributions. First, we integrated MDLM-style and BD3LM-style drafters into an S^{2}D inference engine and measured why they do not provide useful speedups in this setting. Second, we implemented a DFlash hybrid in which DFlash serves S^{2}D speculation-cache misses using fresh verifier activations, while the normal AR S^{2}D drafter serves cache hits and prepares the future tree cache. Third, we identify the main systems bottleneck: diffusion drafting is useful only when the verifier is sufficiently slower than the draft path to amortize cross-GPU communication, scheduling, tree-cache construction, and fallback logic. On small verifier models, those overheads erase the advantage. Finally, we outline the larger-model experiment needed to decide whether DFlash+S^{2}D is worthwhile under representative serving conditions.

2. Method

2.1. Background

Speculative decoding. Given prefix $x_{\leq t}$, a draft model M_d proposes $\hat{x}_{t+1:t+k}$ autoregressively. The target model M_t then evaluates the drafted positions in one forward pass and accepts each token according to the standard rejection-sampling rule $\min(1, p_{M_t}(\hat{x}_i)/p_{M_d}(\hat{x}_i))$ [2, 6]. At the first rejection, SD samples a corrected token from the residual distribution. Under this rule, SD changes latency but not the output distribution.

Speculative^2 decoding. S^{2}D keeps the target and draft on separate devices [5]. While the target verifies the current speculation, the draft predicts likely verification outcomes (a, r), where a is the number of accepted draft tokens and r is the recovery or bonus token. It then precomputes continuations for those outcomes and stores them in a speculation cache. A cache hit eliminates the next drafting wait; a cache miss falls back to a synchronous or just-in-time draft.

Discrete diffusion drafters. Masked diffusion language models iteratively replace mask tokens with predictions [7]. Block diffusion adds a block-causal attention pattern, often called staircase attention, so that a sequence is generated block by block while positions inside the current block are denoised in parallel [1]. These models are bidirectional inside the open block, so only fully committed prefix blocks are safe to reuse in a KV cache; the partially filled open block must be recomputed with accepted tokens clamped and unresolved positions masked.

2.2. Traditional block diffusion in S^{2}D

Our first integration adds a `block` draft backend with three samplers: mask-predict [4], remask, and first-hitting. Given a prefix and lookahead k, the backend constructs

$$[x_{\leq t}, \underbrace{\text{MASK}, \ldots, \text{MASK}}_{k}]$$

and runs R refinement forwards. Each forward predicts logits for all masked positions, commits a subset of tokens according to the sampler, and reuses or recomputes the open block as required by the attention pattern. We implemented both full bidirectional attention and staircase attention with prefix-cache reuse for committed blocks.

This is the most direct diffusion replacement for an AR drafter, but it is not the right systems trade-off. In practice, useful drafts require multiple diffusion forwards. The draft cost is therefore not one block pass; it is approximately

$$T_{\text{draft}} \approx R \cdot T_{\text{diffusion\_forward}},$$

plus cache construction and transfer overhead. When R becomes large enough to produce acceptable tokens, this can approach the cost of generating the block token-by-token with a small AR drafter. Since S^{2}D only helps if the draft path can be hidden under target verification, this iterative diffusion cost becomes a bottleneck.

2.3. DFlash-seeded S^{2}D

DFlash addresses the quality-latency problem by conditioning a lightweight block-diffusion drafter on target hidden states [3]. The target verifier already computes rich activations during prefill and verify; DFlash concatenates selected target-layer residual streams, injects those features into the draft model's KV cache, and predicts the next block from a recovery token followed by mask tokens. The DFlash checkpoint we target, `z-lab/Qwen3-8B-DFlash-b16`, has block size 16, so our v1 path uses $k = 15$ speculative draft tokens.

We integrate DFlash with S^{2}D as a miss handler rather than replacing the whole S^{2}D loop. The hybrid works as follows.

1. The verifier runs the target model and returns logits, recovery tokens, and DFlash target activations.
2. The async draft process receives a speculation-cache request keyed by the sequence id, accepted depth, and recovery token.
3. On a cache hit, S^{2}D returns the normal cached AR-draft branch.

4. On a cache miss, DFlash receives the verifier activation backlog, builds $[\text{recovery}, \text{MASK}^{15}]$, and produces a 15-token draft block using the target embedding and LM head.

5. After the exact verifier accepts or rejects the proposed block, the scheduler appends only the verified target activations to the DFlash backlog and crops the DFlash cache back to the verified prefix.

6. In parallel, the existing S^{2}D AR drafter continues constructing the tree cache for future likely verification outcomes.

The important design choice is that DFlash is used only when the S^{2}D tree cache misses. Hits keep the original S^{2}D fast path, where precomputed AR branches are returned immediately. Misses avoid the weakest fallback of ordinary S^{2}D, namely waiting for a full just-in-time AR draft, and instead use the verifier-conditioned DFlash model to provide the next block.

2.4. Correctness and v1 restrictions

Both block diffusion and DFlash are still draft mechanisms: the target verifier remains exact. At greedy temperature zero, matching AR outputs token-for-token is the correctness test. For stochastic decoding, the same SD residual rule would preserve the target distribution, but our implementation currently focuses on greedy decoding to isolate systems behavior.

The current DFlash+S^{2}D prototype intentionally supports only batch size 1, two GPUs, Qwen3-style target activations, and greedy decoding. The verifier and drafter are placed on separate visible GPUs. This is sufficient to test the central systems question: can a one-pass target-conditioned diffusion block drafter make S^{2}D cache misses cheaper than AR fallback?

3. Experiments

3.1. Setup

Reference SSD evaluation. The original S^{2}D paper evaluates batch-size-one greedy decoding with the target on $4\times$H100 and, for SSD, the asynchronous draft on a separate H100 [5]. Its Llama setup uses Llama-3.1-70B-Instruct with a Llama-3.2-1B draft model; the Qwen appendix uses Qwen3-32B with a Qwen3-0.6B draft model [8]. The benchmark pool contains 128 prompts from each of HumanEval, Alpaca, GSM8K, and UltraFeedback, for 512 prompts total, with 512 output tokens per prompt.

Our local evaluation. Our diffusion experiments were constrained by available compute. We used shared A6000-class machines for implementation and smoke testing, and several larger runs were blocked by other processes occupying most GPU memory. The reported prototype numbers should therefore be read as diagnostic measurements, not as final throughput claims. This limitation matters: S^{2}D is a latency-hiding method, so evaluating it on a small verifier or on busy GPUs can make communication and scheduler overhead look much worse than it would with a larger target.

Table 1. Published S^{2}D results from Kumar et al. [5]. Values are end-to-end decode throughput in tokens/s averaged over four datasets.

Model pair	AR	SD	SSD	SSD/AR
Llama 70B / 1B	54.7	161.8	255.8	4.68×
Qwen3 32B / 0.6B	88.8	136.8	203.8	2.29×

Models. For the traditional block-diffusion path, we used the small Qwen-family stack available in the repository and the block/MDLM-style draft checkpoints supported by our backend. For the DFlash hybrid, we used Qwen3-8B as the target, Qwen3-0.6B as the normal S^{2}D AR drafter, and `z-lab/Qwen3-8B-DFlash-b16` as the target-conditioned DFlash drafter. The DFlash checkpoint has block size 16, so all DFlash runs use $k = 15$.

Metrics. We report end-to-end generated-token throughput, average accepted length per verify step, S^{2}D cache hit rate, DFlash seed latency, and SSD tree-decode latency. Throughput is the user-visible metric. Acceptance and cache hit rate diagnose draft quality and S^{2}D prediction quality. Seed and tree-decode latencies identify whether the bottleneck is DFlash, the AR tree cache, or cross-process orchestration.

3.2. Reference numbers from SSD and DFlash

Table 1 summarizes the published S^{2}D numbers most relevant to this project. The Qwen3-32B results are especially important: they show that S^{2}D still improves over ordinary SD, but the speedup over AR is smaller than in the Llama-70B setting because the verifier is faster and the draft/communication overheads are less easily amortized.

Table 2 gives the DFlash reference point. DFlash is not merely a generic block-diffusion model; it uses target activations and generates the whole block in one conditioned draft pass. Published SGLang results on a single B200 show large Qwen3-8B speedups, which is why DFlash is a more promising diffusion substrate for S^{2}D than iterative block diffusion.

Table 2. Published DFlash SGLang throughput for Qwen3-8B on a single B200 with FlashAttention-4 [3].

Task	AR	DFlash	Speedup	Avg. τ
Math500	230	1175	5.1×	8.01
HumanEval	229	955	4.2×	6.50

Table 3. Current DFlash+S^{2}D prototype diagnostics. This run used Qwen3-8B target, Qwen3-0.6B AR drafter, and `Qwen3-8B-DFlash-b16`; it was a small diagnostic run, not the full 512-prompt evaluation.

Metric	Value
Throughput	11.68 tok/s
Average accepted length	3.30 tokens
S^{2}D cache hit rate	0.60
DFlash seed latency	34.48 ms
SSD tree-decode latency	79 ms

3.3. Our findings

Traditional block diffusion is too slow for SSD. The direct block-diffusion integration did not produce useful S^{2}D speedups. The reason is mechanical: although a diffusion forward predicts all block positions in parallel, the sampler must run many refinement forwards to get acceptable token quality. In the settings where generations were coherent enough to verify well, the number of diffusion passes was close to the number of useful block tokens. The result was a draft-side bottleneck rather than a hidden draft cost. Our best traditional block-diffusion throughput was only about 20 tokens/s, far below the AR and SSD reference points in Table 1.

DFlash improves the diffusion primitive but not yet the full hybrid. The DFlash path removes the main flaw above: it uses fresh target activations and proposes the block in one draft call. Our current hybrid, however, is not yet competitive end to end. On the Qwen3-8B/Qwen3-0.6B/DFlash-b16 prototype, a 32-token smoke run measured:

These numbers should not be interpreted as evidence that DFlash is slow. Rather, they show that our first hybrid pays too much systems overhead for the small verifier used in the test. The DFlash seed itself is tens of milliseconds, while the SSD tree-decode path is even larger. With an 8B verifier on busy A6000s, target verification is not slow enough for the separate-GPU draft path, tensor transfers, cache bookkeeping, and tree construction to be hidden.

Small verifiers are the wrong stress test. S^{2}D benefits when the target verifier is much slower than the draft path. If the verifier is small, the latency gap between verifier and drafter shrinks and fixed costs dominate: inter-GPU tensor transfers, process synchronization, PyTorch dispatch, CUDA graph warmup, cache-key matching, and tree-cache population. This explains why the Qwen3-32B SSD speedup reported by Kumar et al. [5] is lower than the Llama-70B speedup, and why our Qwen3-8B prototype is an especially unfavorable testbed.

Compute limited the representative test. The decisive experiment is DFlash+S^{2}D on a larger verifier, such as Qwen3-32B, Qwen3-Coder-30B-A3B, Qwen3.6-35B-A3B, or Llama-70B, with the DFlash and AR draft paths isolated on a separate GPU. We were not able to run that full experiment within the available GPU credits and shared-server availability. This is a practical limitation, not just an engineering inconvenience: the hypothesis specifically concerns latency hiding at large target-model scale, and small smoke runs cannot validate it.

3.4. Discussion

The negative result is still useful. A generic block-diffusion model is not a drop-in win for S^{2}D because iterative denoising is too expensive. DFlash is the more plausible route because it collapses block drafting to one target-conditioned pass and achieves high acceptance in published serving experiments. Our hybrid design also preserves the strongest part of S^{2}D: cache hits still return precomputed AR branches immediately. The open question is whether DFlash can make misses cheap enough on a verifier large enough that the target pass dominates all other overhead.

4. Conclusion

We investigated whether diffusion drafters are a good fit for Speculative^2 Decoding. The direct approach–replacing the AR drafter with a traditional masked or block-diffusion model–did not work well. Although block diffusion predicts many positions per forward pass, practical generation still requires many refinement passes. In our system this made the diffusion drafter, not the verifier, the bottleneck, with throughput around 20 tokens/s.

DFlash is a better match to the problem because it uses target activations from the verifier and drafts a full block in one conditioned pass. We implemented a DFlash-seeded S^{2}D hybrid in which DFlash serves speculation-cache misses and the normal AR S^{2}D drafter serves cache hits and prepares the future tree cache. This preserves exact verification and keeps the fast S^{2}D hit path intact. Our current Qwen3-8B prototype is not yet fast, reaching 11.68 tokens/s with 34.48 ms DFlash seed latency and 79 ms SSD tree-decode latency, but it isolates the right next bottlenecks.

The broader conclusion is that S^{2}D is sensitive to the verifier/drafter latency ratio. On small target models, cross-GPU communication and bookkeeping overhead can erase any benefit from asynchronous drafting.

Future work should run the DFlash hybrid on a larger verifier–Qwen3-32B, Qwen3-Coder-30B-A3B, Qwen3.6-35B-A3B, or Llama-70B–using enough GPU memory to keep the verifier and draft paths isolated. That experiment requires more compute than was available for this project, but it is the appropriate test of whether DFlash can make S^2D cache misses cheap enough to improve the full throughput-latency frontier.

References

[1] Marianne Arriola, Aaron Gokaslan, Justin T Chiu, Zhihan Yang, Zhixuan Qi, Jiaqi Han, Subham Sekhar Sahoo, and Volodymyr Kuleshov. Block diffusion: Interpolating between autoregressive and diffusion language models. In *ICLR*, 2025. 1, 2

[2] Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*, 2023. 1, 2

[3] Jian Chen, Yesheng Liang, and Zhijian Liu. DFlash: Block diffusion for flash speculative decoding, 2026. 1, 2, 4

[4] Marjan Ghazvininejad, Omer Levy, Yinhan Liu, and Luke Zettlemoyer. Mask-predict: Parallel decoding of conditional masked language models. In *Empirical Methods in Natural Language Processing (EMNLP)*, 2019. 2

[5] Tanishq Kumar, Tri Dao, and Avner May. Speculative speculative decoding, 2026. 1, 2, 3, 4

[6] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning (ICML)*, 2023. 1, 2

[7] Subham Sekhar Sahoo, Marianne Arriola, Yair Schiff, Aaron Gokaslan, Edgar Marroquin, Justin T Chiu, Alexander Rush, and Aditya Grover. Simple and effective masked diffusion language models. In *NeurIPS*, 2024. 1, 2

[8] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, et al. Qwen3 technical report, 2025. 3

[9] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Livia Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. 2